

Microprocessor Systems

Lab6 - 8051 Assembly Language

Prof. Ren-Shuo Liu

NTHU EE



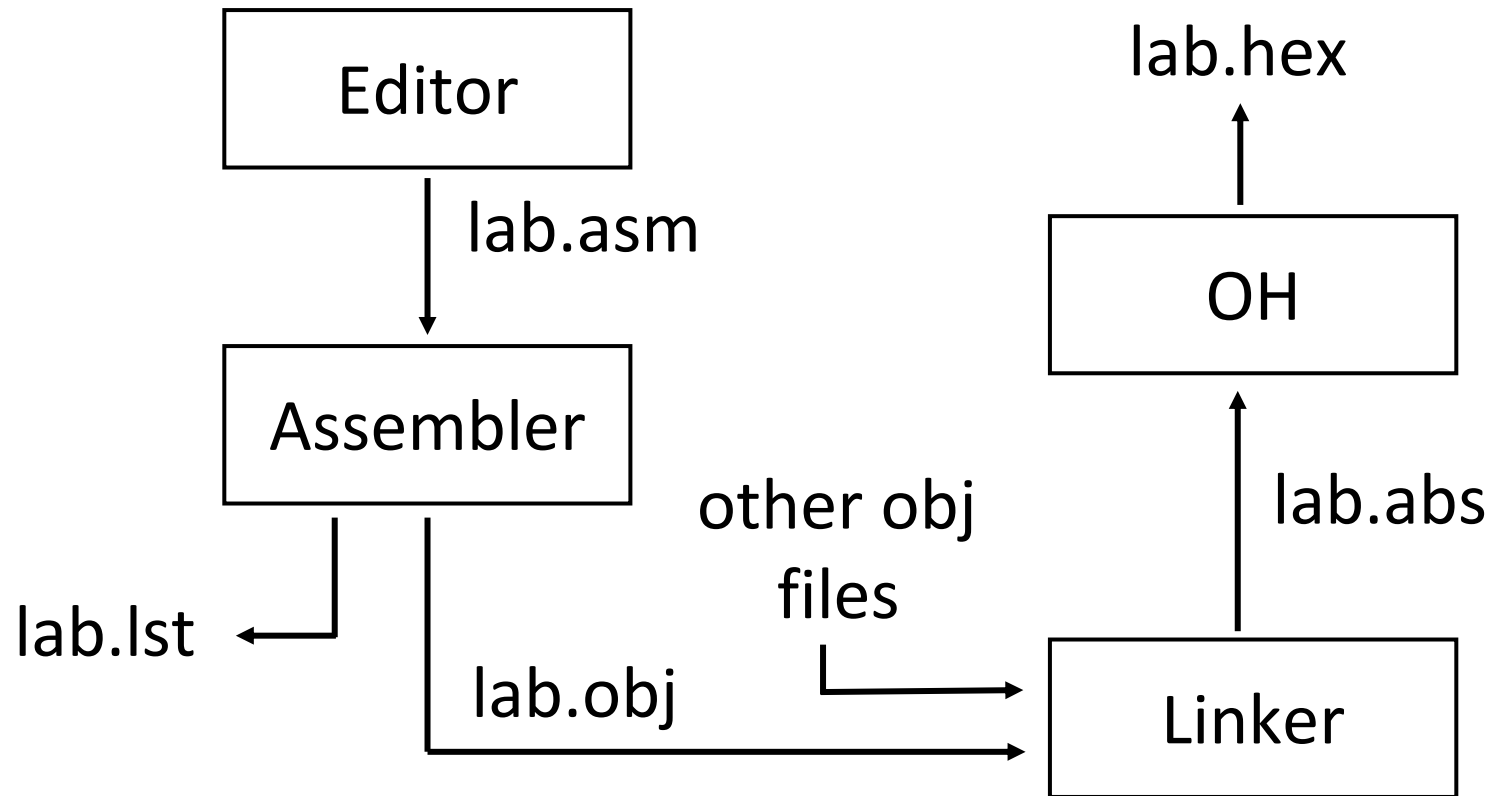
Assembly Program

Label	Instruction	Operand	Comment
-------	-------------	---------	---------

	MOV	A, #01H	; A = 1
Loop:	MOV	P1, A	; P1 = A
	CALL	Delay	
	RR	A	; rotate right
	JMP	Loop	

Delay:

Assembly Language Programming



Instruction Set Architecture

- Instruction Set Architecture (ISA)
- Computers can be divided in to two categories according to their ISA
 - CISC (**complex** instruction set computer)
 - RISC (**reduced** instruction set computer)
- CISC
 - Examples: x86, 8051
- RISC
 - Examples: ARM, MIPS, RISC-V

Instruction Set Architecture

- CISC
 - Legacy computer design philosophy
 - Variable instruction length and cycles
 - **Complicated addressing modes**
 - Complicated combinations of source address and destination address
 - Advantageous in smaller code size
 - Disadvantageous in hardware speed

Instruction Set Architecture

- RISC
 - Modern computer design philosophy
 - Register-register architecture
 - Only data in registers (e.g., R0~R31) can be computed by an instruction
 - Data in memory cannot be computed by an instruction
 - Compute results are stored in registers
 - Load-store
 - Data are transferred from memory to register using **load** instructions
 - Data are transferred from register to memory using **store** instructions
 - Advantageous in high speed and low complexity

8051 Instructions

DATA TRANSFER	ARITHMETIC	LOGICAL	BOOLEAN	BRANCHING
MOV	ADD	ANL	CLR	LJMP
MOVC	ADDC	ORL	SETB	AJMP
MOVB	SUBB	XRL	MOV	SJMP
PUSH	INC	CLR	JC	JZ
POP	DEC	CPL	JNC	JNZ
XCH	MUL	RL	JB	CJNE
XCHD	DIV	RLC	JNB	DJNZ
	DA	RR	JBC	NOP
		RRC	ANL	LCALL
		SWAP	ORL	ACALL
			CPL	RET
• 5 categories of 49 assembly instructions				RETI
				JMP

Register Map (00 – 7F)

Byte address

2F	7F	7E	7D	7C	7B	7A	79	78
2E	77	76	75	74	73	72	71	70
2D	6F	6E	6D	6C	6B	6A	69	68
2C	67	66	65	64	63	62	61	60
2B	5F	5E	5D	5C	5B	5A	59	58
2A	57	56	55	54	53	52	51	50
29	4F	4E	4D	4C	4B	4A	49	48
28	47	46	45	44	43	42	41	40
27	3F	3E	3D	3C	3B	3A	39	38
26	37	36	35	34	33	32	31	30
25	2F	2E	2D	2C	2B	2A	29	28
24	27	26	25	24	23	22	21	20
23	1F	1E	1D	1C	1B	1A	19	18
22	17	16	15	14	13	12	11	10
21	0F	0E	0D	0C	0B	0A	09	08
20	07	06	05	04	03	02	01	00

Bit-addressable locations

7F
30
2F
20
1F
18
17
10
0F
08
07
00

General
purpose
RAM

Bank 3

Bank 2

Bank 1

Bank 0

Register Map (80 – FF)

			Byte address	Bit address	
99	not bit addressable	SBUF			
98	9F 9E 9D 9C 9B 9A 99 98	SCON	FF		
			F0	F7 F6 F5 F4 F3 F2 F1 F0	B
90	97 96 95 94 93 92 91 90	P1			
			E0	E7 E6 E5 E4 E3 E2 E1 E0	ACC
8D	not bit addressable	TH1			
8C	not bit addressable	TH0			
8B	not bit addressable	TL1	D0	D7 D6 D5 D4 D3 D2 D1 D0	PSW
8A	not bit addressable	TL0			
89	not bit addressable	TMOD	B8	- - - BC BB BA B9 B8	IP
88	8F 8E 8D 8C 8B 8A 89 88	TCON			
87	not bit addressable	PCON	B0	B7 B6 B5 B4 B3 B2 B1 B0	P3
83	not bit addressable	DPH	A8	AF - - AC AB AA A9 A8	IE
82	not bit addressable	DPL			
81	not bit addressable	SP			
80	87 86 85 84 83 82 81 80	P0	A0	A7 A6 A5 A4 A3 A2 A1 A0	P2

Registers

- A (Accumulator)
 - Used for all arithmetic and logic operations
- B Register
 - Used for MUL and DIV operations
 - MUL AB, DIV AB
- SP (stack pointer)
 - **Push** increments the SP and writes data
 - **POP** reads data and then decrements the SP
 - Stack is kept in the internal RAM and is restricted to 128 bytes
 - Top most address of stack is “7F”

Registers

- PSW (Program Status Word)

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
CY	AC	F0	RS1	RS0	OV	UD	P

- CY : Carry flag
- AC : Auxiliary Carry flag (For BCD operations)
- F0 : Flag 0 (Available to the user for General purpose)
- (RS1,RS0) : Register bank select
(0,0): Bank0; (0,1): Bank1; (1,0): Bank2; (1,1): Bank3
- OV : Overflow flag
- UD : User definable flag
- P : Parity flag

Registers

- PC (Program Counter)
 - PC is a 2-byte address
 - It tells the 8051 where the next instruction to execute
- DPTR (Data Pointer)
 - The Data Pointer (DPTR) is the 8051's only user-accessible 16-bit (2-byte) register
 - Meant for pointing to data
 - Used by the 8051 to access external memory

Register Bank

RS1 (PSW[4])	RS0 (PSW[3])	Register Bank	Register	Address Range
0	0	0	R0 ~ R7	00H~07H
0	1	1	R0 ~ R7	08H~0FH
1	0	2	R0 ~ R7	10H~17H
1	1	3	R0 ~ R7	18H~1FH

Data Representation

- Decimal
 - Ex: 12
- Binary
 - Ex: 00001100B
- Hexadecimal
 - Ex: 12H , 0E9H

8051 Assembly Code Format

Label	Mnemonic	Destination Operand	Source Operand	Comment
LOOP :	MOV	R0	R1	; for loop

- Label and Comment are not necessary

Addressing Modes

- CPU can access the data in different ways by using addressing modes
- 8051 microcontroller consists of five addressing modes
 - **Direct** addressing
 - **Indirect** addressing
 - **Register** addressing
 - **Immediate** addressing
 - **Index** addressing

Addressing Modes

- **Direct** memory address
- **Register-indirect** memory address
- **Register** name
- **Immediate** constant value
- **Indexed** table lookup

Addressing Modes

- **Direct** memory address
 - MOV A, 10H
- **Register-indirect** memory address
 - MOV A, @R1
- **Register** name
 - MOV A, R5
- **Immediate** constant value
 - MOV A, #10H
- **Indexed** table lookup
 - MOVC A, @A+DPTR

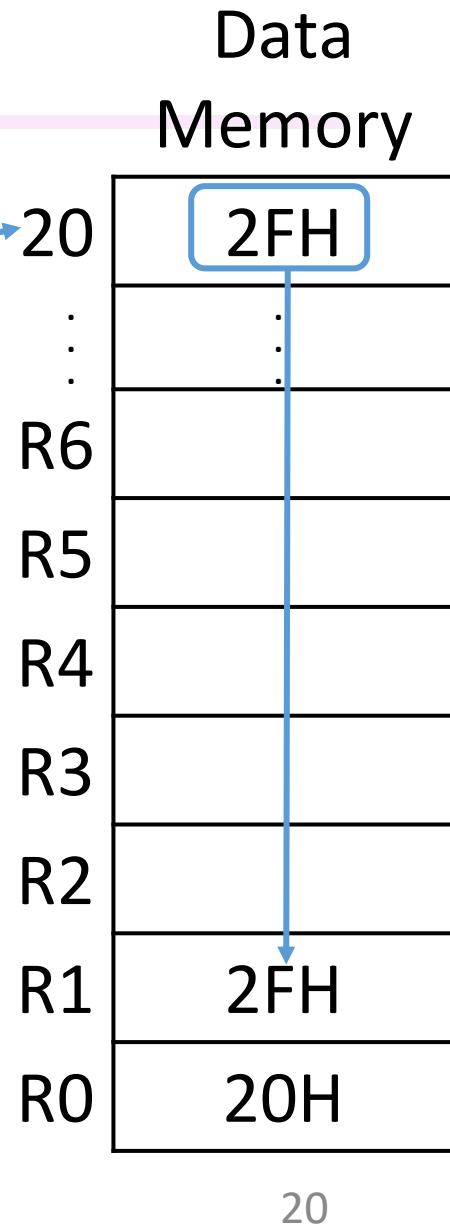
Addressing Modes

- **Direct** memory address
 - MOV A, 10H ; Reg[A] = Data[10]
- **Register-indirect** memory address
 - MOV A, @R1 ; Reg[A] = Data[Reg[R1]]
- **Register** name
 - MOV A, R5 ; Reg[A] = Reg[R5]
- **Immediate** constant value
 - MOV A, #10H ; Reg[A] = 10
- **Indexed** table lookup
 - MOVC A, @A+DPTR
; Reg[A] = Code[Reg[A]+Reg[DPTR]]

Direct Addressing

- MOV Reg, direct
- For example,
MOV R1, 20H

In C language,
`R1 = (*(unsigned char *) 0x20);`

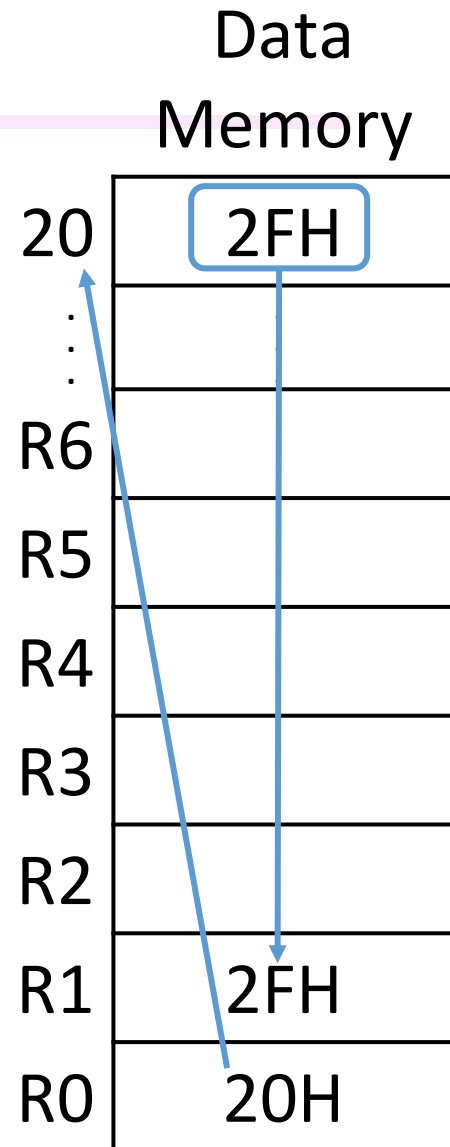


Indirect Addressing

Instruction	Bytes	Cycles
MOV Reg, @Reg	1	1

- Ex: MOV R1, @R0

In C Language,
 $R1 = *R0;$



Register Addressing

Instruction	Bytes	Cycles
MOV Reg, Reg	1	1

- Ex: MOV R1, R0

In C language,
R1 = R0;

Data Memory	
20	2FH
:	:
R6	
R5	
R4	
R3	
R2	
R1	20H
R0	20H

Immediate Addressing

Instruction	Bytes	Cycles
MOV A, #immed	2	1

- Ex: MOV R1, #12H

C Language,
R1 = 0x12;

Data Memory	
20	2FH
:	:
R6	
R5	
R4	
R3	
R2	
R1	12H
R0	20H

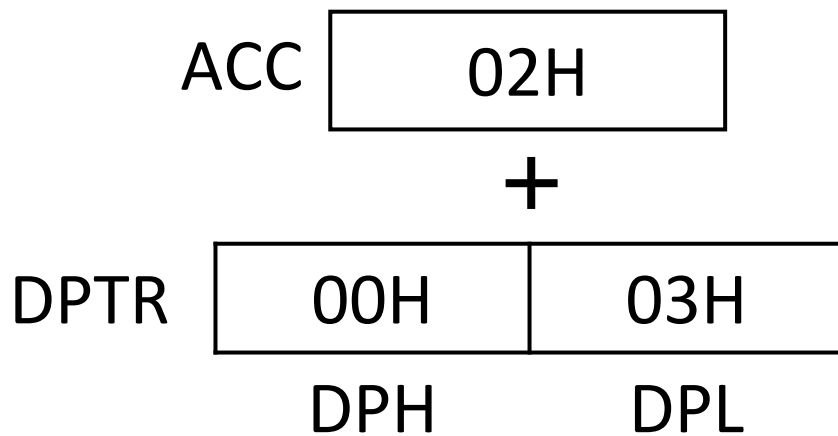
Index Addressing

Instruction	Bytes	Cycles
MOVC Reg, @Reg+Reg	1	2

Instruction	Bytes	Cycles
MOVC Reg, @Reg+Reg	1	2

Index Addressing

- Ex: `MOVC A, @A+DPTR`

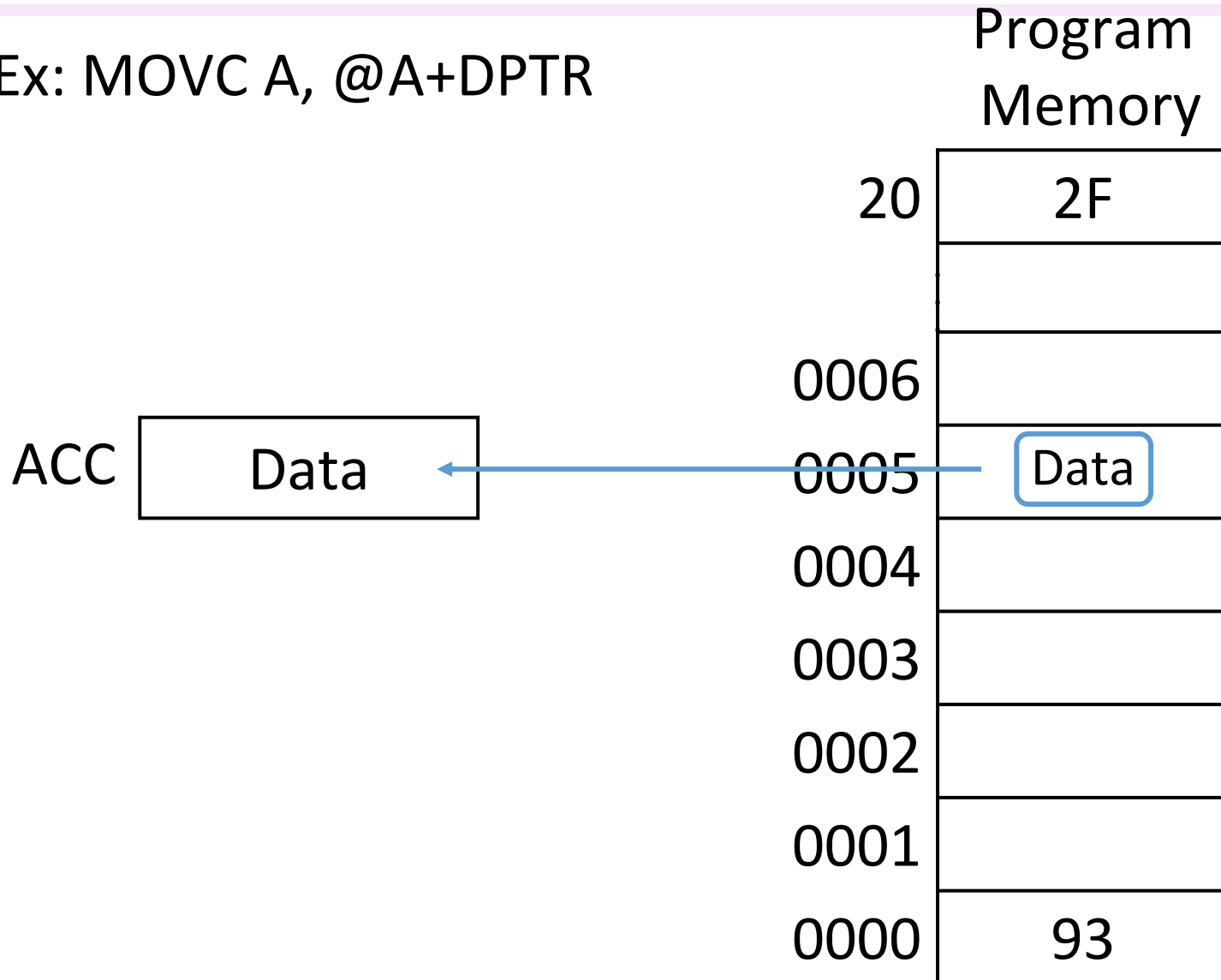


Program Memory

20	2F
0006	
0005	Data
0004	
0003	
0002	
0001	
0000	93

Index Addressing

- Ex: `MOVC A, @A+DPTR`



Instruction Set

- Arithmetic operations
- Logical operations
- Data transfer
- Boolean variable manipulation
- Program branching

Arithmetic Operations

Arithmetic Operations	Description	Byte	Machine Cycle
	Add register to Accumulator	1	1
ADD A, direct	Add direct byte to Accumulator	2	1
	Add indirect RAM to Accumulator	1	1
ADD A, #data	Add immediate data to Accumulator	2	1

Arithmetic Operations

Arithmetic Operations	Description	Byte	Machine Cycle
	Add register to Accumulator with Carry	1	1
	Subtract Register from ACC with borrow	1	1
INC A	Increment Accumulator	1	1
DEC A	Decrement Accumulator	1	1

Arithmetic Operations

Arithmetic Operations	Description	Byte	Machine Cycle
MUL AB	Multiply A & B	1	4
DIV AB	Divide A by B	1	4

Logical Operations

Arithmetic Operations	Description	Byte	Machine Cycle
	AND register to Accumulator	1	1
ANL A, direct	AND direct byte to Accumulator	2	1
	AND indirect RAM to Accumulator	1	1
ANL A, #data	AND immediate data to Accumulator	2	1

Logical Operations

Arithmetic Operations	Description	Byte	Machine Cycle
	OR register to Accumulator	1	1
ORL A, direct	OR direct byte to Accumulator	2	1
	OR indirect RAM to Accumulator	1	1
ORL A, #data	OR immediate data to Accumulator	2	1

Logical Operations

Arithmetic Operations	Description	Byte	Machine Cycle
	Exclusive-OR Register to Accumulator	1	1
XRL A, direct	Exclusive-OR direct byte to Accumulator	2	1
	Exclusive-OR indirect RAM to Accumulator	1	1
XRL A, #data	Exclusive-OR immediate data to Accumulator	2	1

Logical Operations

Arithmetic Operations	Description	Byte	Machine Cycle
CLR A	Clear Accumulator	1	1
CPL A	Complement Accumulator	1	1
RL A	Rotate Accumulator left	1	1
RR A	Rotate Accumulator right	1	1
SWAP A	Swap nibbles within the Accumulator	1	1

Data Transfer

Arithmetic Operations	Description	Byte	Machine Cycle
PUSH direct	Push direct byte onto stack	2	2
POP direct	Pop direct byte from stack	2	2

Boolean Variable Manipulation

Arithmetic Operations	Description	Byte	Machine Cycle
CLR bit	Clear direct bit	2	1
SETB bit	Set direct bit	2	1
CPL bit	Complement direct bit	2	1

Program Branching

Arithmetic Operations	Description	Byte	Machine Cycle
ACALL addr11	Absolute subroutine call	2	2
JZ rel	Jump if Accumulator is zero	2	2
JNZ rel	Jump if Accumulator is not zero	2	2
CJNE A, direct, rel	Compare direct byte to ACC and Jump is not equal	3	2

Program Branching

Arithmetic Operations	Description	Byte	Machine Cycle
	Decrement register and Jump if not zero	2	2
RET	Return from subroutine	1	2

Delay Example

- Since the oscillator frequency is 11.0592Mhz, oscillator period is $1/11.0592$
- In 8051, one machine cycle is 12 oscillator periods
- Machine cycle time approximately equals to $1/11.0592 * 12 = 1.085\mu s$
- In this way, we may know program execution time more accurately

Delay Example Do

Table of Musical Frequencies

Note	Frequency	Note	Frequency	Note	Frequency	Note	Frequency
C	130.82	C	261.63	C	523.25	C	1046.5
C#	138.59	C#	277.18	C#	554.37	C#	1108.73
D	146.83	D	293.66	D	587.33	D	1174.66
D#	155.56	D#	311.13	D#	622.25	D#	1244.51
E	164.81	E	329.63	E	659.26	E	1318.51
F	174.61	F	349.23	F	698.46	F	1396.91
F#	185	F#	369.99	F#	739.99	F#	1479.98
G	196	G	392	G	783.99	G	1567.98
G#	207.65	G#	415.3	G#	830.61	G#	1661.22
A	220	A	440	A	880	A	1760
A#	233.08	A#	466.16	A#	932.33	A#	1864.66
B	246.94	B	493.88	B	987.77	B	1975.53
						C	2093.00

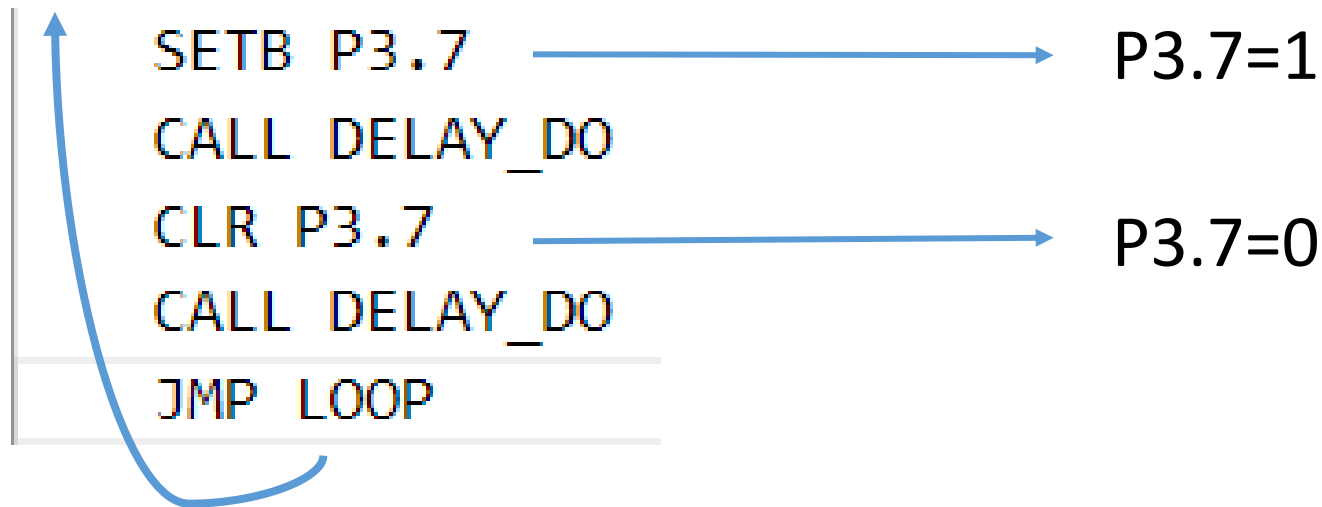
- Note C4 261.63hz (Do)
- Period is $1/261.63=3.822\text{ms}$

Delay Example Do

ORG 0 → Origin, start at memory address 0

START:

LOOP:



Jump to label LOOP

Delay Example Do

```
DELAY_Do:    ;262HZ, We need Delay 1.908ms
             ;1758cycles @Fosc = 11.0592MHz
             MOV R6, #10
             ;+1cycle
DELAY_Do_DL1:
             MOV R7, #80
             DJNZ R7, $
             DJNZ R6, DELAY_Do_DL1
             ;+1630cycle = 10 * (1 + (80*2) + 2)

             MOV R7, #62
             DJNZ R7, $
             ;+125cycle = 1 + (62*2)
             RET    ;+2cycles => Total = 1758
END
```

8051 Instruction Set Manual

- Assembly code

```
ORG 0030H           ;origin start
START:
LOOP:
    MOV R4,#25
DOAGAIN:    MOV R5,#25
DO:
    SETB P3.7
    ACALL DELAY_D0
```

	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
0000 :	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
0010 :	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
0020 :	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
0030 :	7C	19	7D	19	D2	B7	11	62	C2	B7	11	62	DD	F6	DC	F2
0040 :	7C	19	7D	19	D2	B7	11	6F	C2	B7	11	6F	DD	F6	DC	F2
0050 :	7C	19	7D	19	D2	B7	11	7C	C2	B7	11	7C	DD	F6	DC	F2
0060 :	80	27	7E	0A	7F	13	DF	FE	DE	FA	7F	0D	DF	FE	22	7E
0070 :	0A	7F	11	DF	FE	DE	FA	7F	09	DF	FE	22	7E	0A	7F	0F
0080 :	DF	FE	DE	FA	7F	07	DF	FE	22	80	FE	FF	FF	FF	FF	FF
0090 :	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
00A0 :	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
00B0 :	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
00C0 :	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
00D0 :	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF

- Instruction memory address starts at ORG 0030H
- Labels don't occupy memory spaces

8051 Instruction Set Manual

- Assembly code

```
ORG 0030H                ;origin start
START:
LOOP:
DOAGAIN:  MOV R4, #25
DO:       MOV R5, #25
          SETB P3.7
          ACALL DELAY_D0
```

	00	01	02	03
0000	: FF	FF	FF	FF
0010	: FF	FF	FF	FF
0020	: FF	FF	FF	FF
0030	: 7C	19	7D	19
0040	: 7C	19	7D	19
0050	: 7C	19	7D	19

- 2 Bytes long instruction
 - Hex code 7C stands for MOV R4,#immed
 - 25d=19h
 - Hex code 7D stands for MOV R5,#immed

78	2	MOV	R0, #immed
79	2	MOV	R1, #immed
7A	2	MOV	R2, #immed
7B	2	MOV	R3, #immed
7C	2	MOV	R4, #immed
7D	2	MOV	R5, #immed
7E	2	MOV	R6, #immed
7F	2	MOV	R7, #immed

Example (LED Shift)

```
const unsigned char hex_file[] =
{
    0x74, 0xFE, 0xF5, 0x90, 0x03, 0x7D, 0x03, 0x12, 0x00, 0x0C, 0x80, 0xF6, 0x7E, 0xC8, 0x7F, 0xE6,
    0xDF, 0xFE, 0xDE, 0xFA, 0xDD, 0xF6, 0x22
};
```

- **0x74, 0xFE**: MOV A, #FEH
 - A = 0xFE
- **0xF5, 0x90**: MOV P1, A // LED: 0xFE
 - 0x90 is P1 (LED)
 - P1 = A (P1 = 0xFE)
- **0x03**: RR A
 - A = A >> 1 (A = 0x7F)

Example (LED Shift)

```
const unsigned char hex_file[] =  
{  
    0x74, 0xFE, 0xF5, 0x90, 0x03, 0x7D, 0x03, 0x12, 0x00, 0x0C, 0x80, 0xF6, 0x7E, 0xC8, 0x7F, 0xE6,  
    0xDF, 0xFE, 0xDE, 0xFA, 0xDD, 0xF6, 0x22  
};
```

addr16

- 0x7D, 0x03: MOV R5, #03H
 - R5 = 0x03
- 0x12, 0x00, 0x0C: LCALL addr16 // delay function
 - PC = PC + 3
 - SP = SP + 1
 - (SP) = PC[7:0]
 - SP = SP + 1
 - (SP) = PC[15:8]
 - PC = addr16

Example (LED Shift)

```
const unsigned char hex_file[] =  
{  
    0x74, 0xFE, 0xF5, 0x90, 0x03, 0x7D, 0x03, 0x12, 0x00, 0x0C, 0x80, 0xF6, 0x7E, 0xC8, 0x7F, 0xE6,  
    0xDF, 0xFE, 0xDE, 0xFA, 0xDD, 0xF6, 0x22  
};
```

- 0x7E, 0xC8: MOV R6, #C8H
 - R6 = 0xC8
- 0x7F, 0xE6: MOV R7, #E6H
 - R7 = 0xE6
- 0xDF, 0xFE: DJNZ R7, offset
 - PC = PC + 2
 - R7 = R7 - 1
 - if (R7 != 0) PC = PC + offset (offset = 0xFE)

Example (LED Shift)

```
const unsigned char hex_file[] =  
{  
    0x74, 0xFE, 0xF5, 0x90, 0x03, 0x7D, 0x03, 0x12, 0x00, 0x0C, 0x80, 0xF6, 0x7E, 0xC8, 0x7F, 0xE6,  
    0xDF, 0xFE, 0xDE, 0xFA, 0xDD, 0xF6, 0x22  
};
```

- **0xDE, 0xFA**: DJNZ R6, offset
 - $PC = PC + 2$
 - $R6 = R6 - 1$
 - if ($R6 \neq 0$) $PC = PC + \text{offset}$ (offset = 0xFA)
- **0xDD, 0xF6**: DJNZ R5, offset
 - $PC = PC + 2$
 - $R5 = R5 - 1$
 - if ($R6 \neq 0$) $PC = PC + \text{offset}$ (offset = 0xF6)

Example (LED Shift)

```
const unsigned char hex_file[] =
{
    0x74, 0xFE, 0xF5, 0x90, 0x03, 0x7D, 0x03, 0x12, 0x00, 0x0C, 0x80, 0xF6, 0x7E, 0xC8, 0x7F, 0xE6,
    0xDF, 0xFE, 0xDE, 0xFA, 0xDD, 0xF6, 0x22
};
```

- **0x22**: RET // return
 - $PC[15:8] = (SP)$
 - $SP = SP - 1$
 - $PC[7:0] = (SP)$
 - $SP = SP - 1$
- **0x80, 0xF6**: SJMP offset
 - $PC = PC + 2$
 - $PC = PC + \text{offset}$ (offset = 0xF6)
- **0xF5, 0x90**: MOV P1, A // LED shift

Instruction Set Manual

- The other hex code of instructions can be found in the link below
- https://www.keil.com/support/man/docs/is51/is51_opcodes.htm

Reference

- <http://ww1.microchip.com/downloads/en/DeviceDoc/doc4316.pdf>
- https://www.keil.com/support/man/docs/is51/is51_opcodes.htm
- <https://www.electronicshub.org/8051-microcontroller-assembly-language-programming/>
- <https://www.elprocus.com/know-about-types-of-registers-in-8051-microcontroller/>